



# AMISE MedFlow EMR

## Cross-Application Engineering Audit

Prepared: 23 June 2026

Amise Medical Services — Saint Lucia

Dr Dawit Daniel Kabiye, MD, DM

CONFIDENTIAL — FOR INTERNAL USE ONLY

# AMISE MedFlow EMR — Cross-Application Engineering Audit

---

**Date:** 2026-06-23 **Prepared for:** Dr Dawit Daniel Kabiye, MD, DM — Amise Medical Services, Saint Lucia **Classification:** Internal — Engineering Review

---

## Executive Summary

---

This audit examined the full data flow from patient-facing intake (front-desk Next.js app on Vercel) through Supabase persistence to the staff dashboard (React/Vite app), evaluating patient safety, data integrity, and usability across all three applications. The review identified **15 issues** spanning critical patient safety gaps, data integrity failures, and user experience deficiencies. The most urgent findings include missing staff alerts when red-flag symptoms are detected via web intake, no linkage between appointment requests and questionnaire sessions, and silent data loss in the questionnaire save pipeline caused by a database constraint mismatch.

---

## System Architecture

---

The MedFlow EMR platform comprises six interconnected components:

| Component             | Technology                  | Deployment            | Role                                                                                                                                                            |
|-----------------------|-----------------------------|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Front-desk app</b> | Next.js 14, App Router      | Vercel                | Patient-facing intake at <code>/intake</code> — collects chief complaint, referral information, patient details, consent, and adaptive questionnaire responses  |
| <b>API server</b>     | Express 5, Node.js 24       | Render                | Backend for dashboard — booking lifecycle management, notification dispatch (SMS, email), AI intake summarisation, cron jobs                                    |
| <b>Dashboard</b>      | React 19, Vite              | Vercel                | Staff EMR interface — booking inbox, nurse/doctor review queues, questionnaire management, patient records                                                      |
| <b>Supabase</b>       | PostgreSQL, Auth, Storage   | Supabase Cloud        | Persistence layer — 12+ tables with row-level security (RLS), <code>service_role</code> access from API server, <code>anon</code> access from front-desk intake |
| <b>APCQ engine</b>    | TypeScript, shared lib      | Bundled (client-side) | Adaptive Pre-Consultation Questionnaire — client-side scoring, red flag detection, branching question logic in <code>lib/triage-engine</code>                   |
| <b>Communications</b> | Twilio, Gmail API, WhatsApp | Via API server        | Outbound patient notifications gated by <code>MODE env var ( dry_run / supervised / auto )</code> with <code>FORBIDDEN_PATTERNS</code> safety scan              |

#### Key architectural decisions:

- Triage scoring runs entirely in the browser — no API round-trip for acuity calculation.
- All outbound actions (email, SMS, calendar writes) are gated by the `MODE` environment variable.
- Every Claude-drafted reply is scanned against `FORBIDDEN_PATTERNS` before sending; forbidden content (fees, diagnoses, drug doses, results) is quarantined for human review.
- The front-desk app uses Supabase `anon` role with RLS policies for direct database writes during intake, bypassing the API server.

## Data Flow Analysis

### Intake-to-Dashboard Flow



#### Key observations:

1. **Patient opens /intake** — selects chief complaint, completes referral check, enters patient details, provides consent, then answers adaptive APCQ questions driven by the triage engine.

2. **On completion**, `POST /api/intake/submit` creates three records: an `appointment_requests` row (critical — submission fails if this insert fails), plus `questionnaire_sessions` and `questionnaire_responses` rows (best-effort — failures are logged but do not block the submission response).
3. **Dashboard `BookingInboxTab`** fetches pending requests via `GET /api/booking/requests` on the API server. If the API server is unreachable, it falls back to a direct Supabase query for read-only access.
4. **Staff actions** (confirm slot, add to waitlist, cancel) require the API server — there is no Supabase fallback for write operations.

## Findings

### CRITICAL — Patient Safety

#### C1: No staff notification when red flags detected via web intake

|                 |                                                                  |
|-----------------|------------------------------------------------------------------|
| <b>Severity</b> | CRITICAL                                                         |
| <b>Location</b> | <code>artifacts/front-desk/app/api/intake/submit/route.ts</code> |
| <b>Status</b>   | OPEN                                                             |

**Description:** The existing booking flow ( `/api/booking/request` ) sends immediate SMS and email alerts to staff via `STAFF_NOTIFY_PHONE` and `STAFF_NOTIFY_EMAIL` . The APCQ questionnaire intake flow does **not**. When a patient reports red-flag symptoms through the web intake — such as "cannot swallow anything" (emergency severity) or "unintentional weight loss" (urgent severity) — the red flags are recorded in `questionnaire_sessions.red_flags_detected` but generate no proactive notification.

**Impact:** Red-flag symptoms sit silently in the database until a staff member manually checks the nurse review queue. A patient reporting emergency-severity symptoms receives no faster response than one reporting routine heartburn. In a practice environment where queue checks may be periodic, this delay could be clinically significant.

**Recommendation:** Add SMS and email alert dispatch to `STAFF_NOTIFY_PHONE` / `STAFF_NOTIFY_EMAIL` when the APCQ engine determines urgency `>= urgent` at the point of intake submission. The alert should include the patient name, chief complaint, and detected red flags.

**C2: No linkage between appointment\_requests and questionnaire\_sessions**

|                 |                                                                       |
|-----------------|-----------------------------------------------------------------------|
| <b>Severity</b> | <b>CRITICAL</b>                                                       |
| <b>Location</b> | Database schema ( supabase-apcq-migration.sql , supabase-schema.sql ) |
| <b>Status</b>   | OPEN                                                                  |

**Description:** The intake submission creates both an `appointment_requests` row and a `questionnaire_sessions` row independently. There is no foreign key, no shared identifier, and no cross-reference between the two records. Staff viewing a booking in the inbox cannot programmatically trace back to the patient's questionnaire answers.

**Impact:** Matching a booking request to its questionnaire data requires manual correlation by patient name, phone number, and submission timing — a process that is fragile, error-prone, and fails entirely when patients submit multiple requests or when names contain variations. Clinical context captured during the questionnaire (symptom severity, duration, red flags) is effectively disconnected from the booking workflow.

**Recommendation:** Add a `questionnaire_session_id` foreign key column to `appointment_requests`, populated at submission time. Alternatively, store the `appointment_request_id` on the `questionnaire_sessions` record. Ensure the dashboard joins on this FK when displaying booking details.

**C3: AI-generated urgency can override questionnaire-detected red flags**

|                 |                                                                               |
|-----------------|-------------------------------------------------------------------------------|
| <b>Severity</b> | <b>CRITICAL</b>                                                               |
| <b>Location</b> | artifacts/api-server/src/routes/questionnaire.ts<br>( generateIntakeSummary ) |
| <b>Status</b>   | OPEN                                                                          |

**Description:** The `generateIntakeSummary()` function asks Claude to estimate clinical urgency based on the patient's questionnaire responses. The AI-estimated urgency is stored in `intake_summaries.estimated_urgency` and may be used downstream for queue prioritisation. If the APCQ questionnaire engine detected emergency-level red flags but the AI model estimates a lower urgency (e.g., "routine"), the AI assessment could take precedence.

**Impact:** A deterministic, clinically validated red-flag detection could be silently downgraded by a probabilistic AI estimate, potentially delaying care for patients with objectively urgent presentations.

**Recommendation:** Implement `max(questionnaire_severity, ai_severity)` logic — the final urgency should never be lower than what the questionnaire's rule-based engine determined. Log cases where AI and questionnaire urgency diverge for clinical review.

#### C4: Emergency symptoms have no immediate escalation path

|                 |                                                                                          |
|-----------------|------------------------------------------------------------------------------------------|
| <b>Severity</b> | <b>CRITICAL</b>                                                                          |
| <b>Location</b> | APCQ engine ( <code>lib/triage-engine/src/apcq.ts</code> ) and intake submission handler |
| <b>Status</b>   | OPEN                                                                                     |

**Description:** The APCQ engine defines an `emergency` severity level for life-threatening presentations (e.g., complete dysphagia / saliva-only swallowing at line 201 of `apcq.ts` ). When this severity is triggered, no special action occurs — no 911/ambulance guidance is shown to the patient, no immediate doctor notification is sent, and no escalation workflow is initiated. By contrast, the email intake flow ( `intake.ts` lines 45-54) does include escalation logic for high-acuity presentations.

**Impact:** A patient using the web intake who reports symptoms consistent with a surgical emergency receives the same "thank you, we will be in touch" response as any other submission. There is a life safety gap in the web intake path.

**Recommendation:** Create a mandatory escalation flow for `emergency` severity: (1) display immediate guidance to the patient (e.g., "Please call 911 or proceed to the nearest emergency department"), (2) send an immediate SMS/email to the doctor's direct line, and (3) flag the record for priority review in the dashboard.

## HIGH — Data Integrity

#### H1: `delivery_method` check constraint rejected web intake sessions

|                 |                                                                                       |
|-----------------|---------------------------------------------------------------------------------------|
| <b>Severity</b> | <b>HIGH</b> (was causing silent data loss)                                            |
| <b>Location</b> | <code>supabase-apcq-migration.sql</code> lines 180-185, <code>route.ts</code> line 85 |
| <b>Status</b>   | <b>FIXED</b>                                                                          |

**Description:** The `questionnaire_sessions` table's `delivery_method` column had a `CHECK` constraint allowing only `kiosk`, `whatsapp_link`, `qr_code`, and `staff_assisted`. The front-desk intake handler inserted `'web_intake'` — a value not in the constraint. The insert failed silently because questionnaire session creation is best-effort (errors are caught and logged but do not fail the submission).

**Impact:** Every web intake questionnaire session was being dropped. Patient responses to the adaptive questionnaire were lost, leaving only the appointment request with no clinical context.

**Resolution:** Migration `supabase-web-intake-delivery-method-migration.sql` created — adds `'web_intake'` to the allowed values.

---

## H2: QuestionnaireManagerTab fallback queried non-existent columns

|                 |                                                                                      |
|-----------------|--------------------------------------------------------------------------------------|
| <b>Severity</b> | <b>HIGH</b>                                                                          |
| <b>Location</b> | <code>artifacts/dashboard/src/pages/tabs/QuestionnaireManagerTab.tsx</code> line 304 |
| <b>Status</b>   | <b>FIXED</b>                                                                         |

**Description:** The Supabase fallback query in the Questionnaire Manager tab attempted to `SELECT patient_name, template_key, red_flags` — none of which exist as columns on the `questionnaire_sessions` table. The fallback silently returned no results.

**Impact:** When the API server was unavailable, the questionnaire management interface showed an empty queue regardless of actual pending sessions.

**Resolution:** Query updated to use correct column names with appropriate joins to resolve patient names and template information.

---

## H3: source column not explicitly set on intake submissions

|                 |                                                                  |
|-----------------|------------------------------------------------------------------|
| <b>Severity</b> | <b>HIGH</b>                                                      |
| <b>Location</b> | <code>artifacts/front-desk/app/api/intake/submit/route.ts</code> |
| <b>Status</b>   | <b>FIXED</b>                                                     |

**Description:** The front-desk intake submission did not explicitly set the `source` column on `appointment_requests`. The dashboard's source badge rendered blank for all web intake submissions.

**Impact:** Staff could not distinguish web intake submissions from other booking sources in the inbox view.

**Resolution:** Submission handler now explicitly sets `source: 'web'`.

---



**H4: Dashboard enters read-only mode when API server is down**

|                 |                                                                      |
|-----------------|----------------------------------------------------------------------|
| <b>Severity</b> | <b>HIGH</b>                                                          |
| <b>Location</b> | artifacts/dashboard/src/pages/tabs/BookingInboxTab.tsx lines 305-387 |
| <b>Status</b>   | OPEN                                                                 |

**Description:** The Supabase fallback enables the dashboard to load and display booking data when the API server is unreachable. However, all staff actions — Confirm Slot, Add to Waitlist, Cancel — route through the API server exclusively. There is no Supabase-direct fallback for write operations, and no visual indicator that the dashboard is operating in degraded/read-only mode.

**Impact:** Staff can see pending bookings but cannot act on them. Without a degraded-mode indicator, staff may repeatedly attempt actions that silently fail, delaying patient scheduling.

**Recommendation:** (1) Implement Supabase-direct writes for critical booking actions as a fallback. (2) Display a prominent banner when operating in fallback mode: "API server unavailable — limited functionality. Contact support if this persists."

**H5: Nurse review step not enforced before doctor approval**

|                 |                                                                            |
|-----------------|----------------------------------------------------------------------------|
| <b>Severity</b> | <b>HIGH</b>                                                                |
| <b>Location</b> | artifacts/api-server/src/routes/questionnaire.ts (doctor-approve endpoint) |
| <b>Status</b>   | OPEN                                                                       |

**Description:** The `/api/questionnaire/session/:token/doctor-approve` endpoint does not validate that `nurse_reviewed_at` has been set on the questionnaire session. A doctor can approve a session that has never been reviewed by a nurse.

**Impact:** The two-step review workflow (nurse triage then doctor approval) — designed as a safety gate — can be bypassed entirely. Clinical observations that a nurse would flag during review may be missed.

**Recommendation:** Add server-side validation: reject doctor approval requests where `nurse_reviewed_at IS NULL` with a clear error message indicating that nurse review is required first.

**MEDIUM — UX / Compliance**

**M1: No input validation on phone number or email address**

|                 |                                              |
|-----------------|----------------------------------------------|
| <b>Severity</b> | MEDIUM                                       |
| <b>Location</b> | Front-desk intake form, patient details step |
| <b>Status</b>   | OPEN                                         |

**Description:** The phone number field accepts any string (including alphabetic characters and single digits). The email field has no server-side format validation. Client-side validation is minimal.

**Impact:** Invalid contact information renders SMS and email notifications undeliverable, with no feedback to staff until a send attempt fails downstream.

**Recommendation:** Add pattern validation on the client ( `+1758XXXXXX` for Saint Lucia numbers) and server-side format checks in the submission handler.

**M2: Complete data loss on page refresh during intake**

|                 |                                                       |
|-----------------|-------------------------------------------------------|
| <b>Severity</b> | MEDIUM                                                |
| <b>Location</b> | Front-desk intake flow (React <code>useState</code> ) |
| <b>Status</b>   | OPEN                                                  |

**Description:** All intake state — chief complaint selection, patient details, referral information, consent, and questionnaire progress — is held in React `useState` . A page refresh at any point during the multi-step intake flow loses all entered data. Mobile browsers frequently unload background tabs, compounding this issue.

**Impact:** Patients who lose progress mid-intake may abandon the process or re-submit with incomplete information, leading to frustration and potential duplicate records.

**Recommendation:** Persist intake state to `localStorage` or `sessionStorage` after each step transition, with automatic restoration on page load.

**M3: Error state displays "Thank you" message alongside error banner**

|                 |                                      |
|-----------------|--------------------------------------|
| <b>Severity</b> | MEDIUM                               |
| <b>Location</b> | Front-desk intake completion handler |
| <b>Status</b>   | OPEN                                 |

**Description:** The `handleComplete()` function sets the screen to `'complete'` (showing a "Thank you" confirmation) **before** checking the submission result. If submission fails, the patient sees both a success message and an error banner simultaneously.

**Impact:** Patients may believe their submission succeeded when it did not, and may not seek to re-submit.

**Recommendation:** Only transition to the success screen after the submission response confirms success. Show a clear error state with retry option on failure.

#### M4: Referral fields not validated when referral type is "doctor"

|                 |                                 |
|-----------------|---------------------------------|
| <b>Severity</b> | MEDIUM                          |
| <b>Location</b> | Front-desk intake referral step |
| <b>Status</b>   | OPEN                            |

**Description:** When a patient selects `referralType: 'doctor'`, the referring doctor name and institution fields can both be submitted empty.

**Impact:** Referral information required for clinical context and correspondence with referring physicians is missing from the record.

**Recommendation:** Make referring doctor name and institution required fields when the doctor referral type is selected.

#### M5: FORBIDDEN\_PATTERNS not applied to AI-generated intake summaries

|                 |                                                               |
|-----------------|---------------------------------------------------------------|
| <b>Severity</b> | MEDIUM                                                        |
| <b>Location</b> | <code>artifacts/api-server/src/routes/questionnaire.ts</code> |
| <b>Status</b>   | OPEN                                                          |

**Description:** The `FORBIDDEN_PATTERNS` safety scan (which catches diagnoses, fees, medication dosages, and test results) is applied to Claude-drafted outbound emails and SMS messages. It is **not** applied to AI-generated intake summaries produced by `generateIntakeSummary()`.

**Impact:** Intake summaries displayed to staff could contain content that, if inadvertently shared with patients, would violate the practice's communication policy. The risk is lower than for outbound messages but represents an inconsistent application of the safety layer.

**Recommendation:** Apply `FORBIDDEN_PATTERNS` scanning to all AI-generated text, including intake summaries. Flag violations for review rather than blocking, since summaries are staff-facing.

---

#### M6: Accessibility gaps in intake interface

|                 |                                 |
|-----------------|---------------------------------|
| <b>Severity</b> | MEDIUM                          |
| <b>Location</b> | Front-desk intake UI components |
| <b>Status</b>   | OPEN                            |

**Description:** Multiple accessibility issues identified: - Option buttons lack `aria-label` and `role="radio"` attributes - Scale inputs (pain/severity scales) have no associated labels - Secondary text colour ( `#a8a29e` ) achieves approximately 3.5:1 contrast ratio against white background, failing WCAG AA minimum of 4.5:1

**Impact:** The intake form may be unusable or difficult to navigate for patients using screen readers or with visual impairments. Non-compliance with WCAG AA accessibility standards.

**Recommendation:** Conduct a WCAG AA compliance pass: add ARIA roles and labels to interactive elements, associate labels with all form inputs, and increase contrast ratios to meet 4.5:1 minimum.

---

#### M7: `consultation_requests` table may not exist in production

|                 |                                                 |
|-----------------|-------------------------------------------------|
| <b>Severity</b> | MEDIUM                                          |
| <b>Location</b> | <code>ConsultationRequestsView</code> component |
| <b>Status</b>   | OPEN                                            |

**Description:** The `ConsultationRequestsView` component queries a `consultation_requests` table. If the corresponding migration has not been applied to the production database, both the API route and the Supabase fallback query will fail.

**Impact:** The consultation requests page would display an error or empty state with no clear indication that the underlying table is missing.

**Recommendation:** Verify migration status in production. Add graceful error handling that distinguishes "no records" from "table does not exist."

## Required Migrations

---

The following database migrations must be applied to resolve identified issues:

| # | Migration File                                             | Purpose                                                                                      | Status                       |
|---|------------------------------------------------------------|----------------------------------------------------------------------------------------------|------------------------------|
| 1 | supabase-web-intake-delivery-method-migration.sql          | Adds 'web_intake' to delivery_method CHECK constraint on questionnaire_sessions              | Created, pending application |
| 2 | supabase-anon-intake-migration.sql<br>(remaining policies) | Adds anon role INSERT policies for questionnaire_sessions and questionnaire_responses tables | Pending verification         |

---

## Priority Matrix

| ID        | Finding                           | Severity        | Impact                     | Status       | Effort |
|-----------|-----------------------------------|-----------------|----------------------------|--------------|--------|
| <b>C1</b> | No red flag staff alerts          | <b>CRITICAL</b> | Delayed emergency response | OPEN         | Medium |
| <b>C2</b> | No booking-questionnaire linkage  | <b>CRITICAL</b> | Lost clinical context      | OPEN         | Low    |
| <b>C3</b> | AI urgency can override red flags | <b>CRITICAL</b> | Downgraded emergencies     | OPEN         | Low    |
| <b>C4</b> | No emergency escalation path      | <b>CRITICAL</b> | Life safety gap            | OPEN         | Medium |
| <b>H1</b> | delivery_method constraint        | <b>HIGH</b>     | Silent data loss           | <b>FIXED</b> | Done   |
| <b>H2</b> | Fallback column names             | <b>HIGH</b>     | Empty queue display        | <b>FIXED</b> | Done   |
| <b>H3</b> | Missing source column             | <b>HIGH</b>     | Blank source badges        | <b>FIXED</b> | Done   |
| <b>H4</b> | Read-only fallback mode           | <b>HIGH</b>     | Staff cannot act           | OPEN         | Medium |
| <b>H5</b> | Nurse review bypass               | <b>HIGH</b>     | Skipped safety gate        | OPEN         | Low    |
| <b>M1</b> | No phone/email validation         | <b>MEDIUM</b>   | Bad contact data           | OPEN         | Low    |
| <b>M2</b> | State lost on refresh             | <b>MEDIUM</b>   | Patient frustration        | OPEN         | Medium |
| <b>M3</b> | Success + error shown together    | <b>MEDIUM</b>   | Confusing UX               | OPEN         | Low    |
| <b>M4</b> | Empty referral fields             | <b>MEDIUM</b>   | Missing referral info      | OPEN         | Low    |
| <b>M5</b> | FORBIDDEN_PATTERNS gap            | <b>MEDIUM</b>   | Unsafe AI content          | OPEN         | Low    |
| <b>M6</b> | Accessibility                     | <b>MEDIUM</b>   | WCAG non-compliance        | OPEN         | Medium |
| <b>M7</b> | Missing table                     | <b>MEDIUM</b>   | Broken page                | OPEN         | Low    |

**Summary:** 4 critical, 5 high (3 fixed), 7 medium — **11 open items remaining.**

## Recommended Resolution Order

- 1. Immediate (this week):** C1, C4 — Patient safety alerts and emergency escalation. These represent life safety gaps.
- 2. Urgent (within two weeks):** C2, C3 — Booking-questionnaire linkage and urgency override protection. These ensure clinical data integrity.

3. **Short-term (within one month):** H4, H5 — Dashboard resilience and nurse review enforcement.
  4. **Planned (within quarter):** M1 through M7 — Validation, UX, accessibility, and compliance items.
- 

*Report prepared as part of cross-application engineering review. All file paths reference the `amise-medflow-emr-public` repository. Findings should be tracked as issues in the project management system with the severity labels assigned above.*

# Appendix: System Flow Diagrams

---

**Note:** These diagrams are written in Mermaid notation. Render them interactively at [mermaid.live](https://mermaid.live) or in any Mermaid-compatible viewer (GitHub, VS Code, Notion). Red/orange nodes indicate identified issues. Green nodes indicate fixed items.



# AMISE MedFlow EMR — System Diagrams

---

## Diagram 1: Patient Intake Flow (Current State — With Issues Marked)

---

```

graph TD
    Start([Patient Opens Intake Form]) --> CC[Chief Complaint Selection<br/>Multi-select from]
    CC --> RefCheck{Referral Check:<br/>Self or Doctor?}

    RefCheck -->|Self-referred| Details
    RefCheck -->|Doctor-referred| RefUpload

    subgraph Referral Upload
        RefUpload[Referral Upload Screen<br/>Doctor name, institution dropdown,<br/>WhatsApp s]
        RefIssue[/ISSUE: Referral fields not validated<br/>— can be submitted empty/]
        RefUpload --> RefIssue
    end

    end

    RefUpload --> Details

    subgraph Patient Details
        Details[Patient Details Form<br/>name*, phone*, email, DOB]
        PhoneIssue[/ISSUE: No phone format validation/]
        EmailIssue[/ISSUE: No email format validation/]
        Details --> PhoneIssue
        Details --> EmailIssue
    end

    end

    Details --> Consent[Consent Screen<br/>Checkbox acceptance required]
    Consent --> APCQ

    subgraph APCQ Questionnaire
        APCQ[Adaptive APCQ Questions<br/>3–18 questions, branching logic]
        StateIssue[/ISSUE: State not persisted<br/>— refresh loses all data/]
        APCQ --> StateIssue
        APCQ -->|At any step| WAEscape([WhatsApp Escape Hatch])
        APCQ -->|Red flags detected| RedBanner[Red Flag Banner<br/>shown to patient]
        RedBanner --> APCQ
    end

    end

    APCQ -->|Questions complete| Submit[POST /api/intake/submit]

    subgraph Submission Processing
        Submit --> ThankYouEarly
        ThankYouEarly[/ISSUE: Screen shows 'Thank you'<br/>before confirming save/]

        Submit --> AR[Creates appointment_requests row<br/>source: web]
        Submit --> QS[Creates questionnaire_sessions row<br/>best-effort]
        Submit --> QR[Creates questionnaire_responses rows<br/>best-effort]

        QSFIXED[FIXED: delivery_method='web_intake'<br/>was rejected by check constraint]
        QSISSUE[/ISSUE: No FK link from<br/>questionnaire_sessions to<br/>appointment_requests]

        QS --> QSFIXED
    end

```

```

    QS --> QSIssue
end

Submit -->|Success| ThankYou([Thank You + Confirmation])
Submit -->|Failure| ErrorBanner[Error Banner +<br/>WhatsApp Fallback Link]

classDef issue fill:#ff6b6b,stroke:#c0392b,color:#fff,stroke-width:2px
classDef fixed fill:#2ecc71,stroke:#27ae60,color:#fff,stroke-width:2px
classDef normal fill:#3498db,stroke:#2980b9,color:#fff
classDef start fill:#9b59b6,stroke:#8e44ad,color:#fff
classDef escape fill:#f39c12,stroke:#e67e22,color:#fff

class RefIssue,PhoneIssue,EmailIssue,StateIssue,ThankYouEarly,QSIssue issue
class QSFxed fixed
class Start,ThankYou start
class WAEscape,ErrorBanner escape
class CC,RefUpload,Details,Consent,APCQ,Submit,AR,QS,QR,RedBanner normal
class RefCheck normal

```

## Diagram 2: Data Flow — Intake to Dashboard (With Gaps)

---

```

graph LR
  subgraph Patient Side
    Browser([Patient Browser])
  end

  subgraph "Front-Desk App (Next.js on Vercel)"
    SubmitRoute[POST /api/intake/submit]
  end

  subgraph "Supabase Database"
    AR[(appointment_requests<br/>patient_name, phone, email,<br/>appointment_type, location)]
    QS[(questionnaire_sessions)]
    QR[(questionnaire_responses)]
  end

  subgraph "MISSING PATHS"
    NoSMS[/MISSING: No SMS or email<br/>notification to staff on submission/]
    NoAlert[/MISSING: No alert on red flags/]
  end

  Browser --> SubmitRoute

  SubmitRoute -->|INSERT| AR
  SubmitRoute -->|INSERT best-effort| QS
  SubmitRoute -->|INSERT best-effort| QR

  QSIssueA[/FIXED: delivery_method<br/>constraint/]
  QSIssueB[/ISSUE: No link to<br/>appointment_requests/]
  QS --> QSIssueA
  QS --> QSIssueB

  SubmitRoute -->|Should trigger| NoSMS
  SubmitRoute -->|Should trigger| NoAlert

  subgraph "Staff Side"
    StaffBrowser([Staff Browser<br/>Dashboard])
  end

  subgraph "API Server (Express on Render)"
    BookingAPI[GET /api/booking/requests]
  end

  StaffBrowser -->|Primary path| BookingAPI
  BookingAPI -->|Query| AR

  StaffBrowser -->|Fallback when API down| AR
  FallbackIssue1[/ISSUE: Fallback is read-only<br/>- no write actions available/]
  FallbackIssue2[/ISSUE: No visual indicator<br/>of fallback mode/]
  StaffBrowser --> FallbackIssue1

```

```

StaffBrowser -.-> FallbackIssue2

subgraph "Staff Actions (API Server Required)"
  Confirm[Confirm Booking]
  Waitlist[Waitlist Booking]
  Cancel[Cancel Booking]
  ActionIssue[/ISSUE: Actions fail when<br/>API server is down/]
end

StaffBrowser --> Confirm
StaffBrowser --> Waitlist
StaffBrowser --> Cancel
Confirm -.-> ActionIssue
Waitlist -.-> ActionIssue
Cancel -.-> ActionIssue

classDef issue fill:#ff6b6b,stroke:#c0392b,color:#fff,stroke-width:2px
classDef fixed fill:#2ecc71,stroke:#27ae60,color:#fff,stroke-width:2px
classDef missing fill:#e67e22,stroke:#d35400,color:#fff,stroke-width:2px
classDef db fill:#1abc9c,stroke:#16a085,color:#fff
classDef app fill:#3498db,stroke:#2980b9,color:#fff
classDef user fill:#9b59b6,stroke:#8e44ad,color:#fff

class QSIssueB,FallbackIssue1,FallbackIssue2,ActionIssue issue
class QSIssueA fixed
class NoSMS,NoAlert missing
class AR,QS,QR db
class SubmitRoute,BookingAPI,Confirm,Waitlist,Cancel app
class Browser,StaffBrowser user

```

## Diagram 3: Clinical Safety Gates (Current vs Required)

---

```

graph TD
    Submit([Patient Submits Intake]) --> Gap1
    Gap1[Gap: No automatic<br/>staff notification]
    Gap1 --> Queue[Booking Appears in Queue]
    Queue --> Gap2[Gap: No priority sorting by<br/>red flag severity in inbox]
    Gap2 --> StaffReview[Staff Reviews Booking]
    StaffReview --> Gap3[Gap: Cannot see questionnaire<br/>answers from inbox — no linkage]
    Gap3 --> NurseQueue[Nurse Opens<br/>Questionnaire Queue]
    NurseQueue --> NurseReview[Nurse Reviews Session]
    NurseReview --> Gap4[Gap: Nurse review<br/>not enforced]
    Gap4 --> DoctorApprove[Doctor Approves]
    DoctorApprove --> Gap5[Gap: AI urgency can override<br/>questionnaire red flags]
    Gap5 --> EMR([EMR Populated])

    subgraph "Current Flow (with gaps)"
        Gap1
        Queue
        Gap2
        StaffReview
        Gap3
        NurseQueue
        NurseReview
        Gap4
        DoctorApprove
        Gap5
    end

    subgraph "Required Safety Improvements"
        Should1[SHOULD: SMS/email alert on<br/>urgent/emergency submission]:::should1
        Should1 -->|Required| Gap1
        Should2[SHOULD: Mandatory nurse review<br/>before doctor approval]:::should2
        Should2 -->|Required| Gap4
        Should3[SHOULD: Use max of questionnaire<br/>urgency and AI urgency]:::should3
        Should3 -->|Required| Gap5
        Should4[SHOULD: Booking links to<br/>questionnaire session via FK]:::should4
        Should4 -->|Required| Gap3
    end

```



```

    Should2
    Should3
    Should4
end

classDef gap fill:#e74c3c,stroke:#c0392b,color:#fff,stroke-width:2px
classDef should fill:#2ecc71,stroke:#27ae60,color:#fff,stroke-width:2px,stroke-dasharray
classDef normal fill:#3498db,stroke:#2980b9,color:#fff
classDef endpoint fill:#9b59b6,stroke:#8e44ad,color:#fff

class Gap1,Gap2,Gap3,Gap4,Gap5 gap
class Should1,Should2,Should3,Should4 should
class Queue,StaffReview,NurseQueue,NurseReview,DoctorApprove normal
class Submit,EMR endpoint

```

## Diagram 4: System Architecture Overview

---

```

graph TB
  subgraph "Patient-Facing"
    PatientBrowser([Patient Browser])
  end

  subgraph "Staff-Facing"
    StaffBrowser([Staff Browser])
  end

  subgraph "Front-Desk App (Next.js – Vercel)"
    Intake[/intake – APCQ Questionnaire/]
    Refer[/refer – Doctor Referral Form/]
    Book[/book – Booking Request/]
    TokenQ[/questionnaire/token – Token-based Questionnaire/]
  end

  subgraph "Dashboard App (React/Vite – Vercel)"
    BookingInbox[Booking Inbox]
    QuestionnaireQueue[Questionnaire Queue]
    ConsultationReqs[Consultation Requests]
  end

  subgraph "API Server (Express – Render)"
    APIRoutes[Express Routes<br>/api/booking/*<br>/api/intake/*<br>/api/questionnaire/]
  end

  subgraph "Supabase (PostgreSQL + Auth)"
    direction TB
    AppReqs[(appointment_requests)]
    QSessions[(questionnaire_sessions)]
    QResponses[(questionnaire_responses)]
    Patients[(patients)]
    AuditLog[(audit_log)]
    UserProfiles[(user_profiles)]
  end

  subgraph "External Services"
    Claude[Claude AI<br>Intake summaries, triage]
    Twilio[Twilio SMS]
    Gmail[Gmail API]
    GCal[Google Calendar]
  end

  PatientBrowser --> Intake
  PatientBrowser --> Refer
  PatientBrowser --> Book
  PatientBrowser --> TokenQ

  Intake --> |Direct INSERT<br>anon key| AppReqs

```

```

Intake -->|Direct INSERT<br/>anon key| QSessions
Intake -->|Direct INSERT<br/>anon key| QResponses
Refer -->|Direct INSERT<br/>anon key| AppReqs
Book -->|Direct INSERT<br/>anon key| AppReqs

StaffBrowser --> BookingInbox
StaffBrowser --> QuestionnaireQueue
StaffBrowser --> ConsultationReqs

BookingInbox -->|Primary data path| APIRoutes
QuestionnaireQueue -->|Primary data path| APIRoutes
ConsultationReqs -->|Primary data path| APIRoutes

BookingInbox -->|Fallback when API down<br/>READ-ONLY| AppReqs

APIRoutes -->|service_role key| AppReqs
APIRoutes -->|service_role key| QSessions
APIRoutes -->|service_role key| QResponses
APIRoutes -->|service_role key| Patients
APIRoutes -->|service_role key| AuditLog

APIRoutes --> Claude
APIRoutes --> Twilio
APIRoutes --> Gmail
APIRoutes --> GCal

linkStyle 12 stroke:#e74c3c,stroke-width:3px,stroke-dasharray: 5 5

classDef patient fill:#9b59b6,stroke:#8e44ad,color:#fff
classDef staff fill:#2980b9,stroke:#2471a3,color:#fff
classDef frontend fill:#1abc9c,stroke:#16a085,color:#fff
classDef dashboard fill:#3498db,stroke:#2980b9,color:#fff
classDef api fill:#e67e22,stroke:#d35400,color:#fff
classDef db fill:#27ae60,stroke:#229954,color:#fff
classDef external fill:#8e44ad,stroke:#7d3c98,color:#fff
classDef unreliable fill:#e74c3c,stroke:#c0392b,color:#fff

class PatientBrowser patient
class StaffBrowser staff
class Intake,Refer,Book,TokenQ frontend
class BookingInbox,QuestionnaireQueue,ConsultationReqs dashboard
class APIRoutes api
class AppReqs,QSessions,QResponses,Patients,AuditLog,UserProfiles db
class Claude,Twilio,Gmail,GCal external

```